

Executive Summary

In modern RAG (Retrieval-Augmented Generation) systems, standalone LLMs struggle with outdated knowledge and hallucinations. Agentic, multi-agent RAG systems overcome this by connecting LLMs to current, domain-specific data sources and by orchestrating multiple specialized agents [3†L180-L188]

[15†L118-L127] . A production-grade **AI Multi-Agent Research Assistant** would preprocess and index a document corpus (via text-splitting and embeddings), store it in a vector DB (e.g. Weaviate), and use LangChain or similar as an orchestration layer. User queries are routed to an AI “agent” that retrieves relevant context (possibly via multiple retrievers and external tools) and then synthesizes an answer with citations from the retrieved data. Our report covers: problem/use cases, market need and competitors, detailed architecture (with component and data-flow diagrams), ER/workflow Mermaid charts, exact folder structure and DB schema (Postgres + Weaviate), API endpoints, multi-agent design, RAG pipeline details, memory/session management, deployment (Docker, CI/CD, cost for 1000 users), evaluation metrics, security/privacy compliance, a 60-day roadmap, MVP features, and recommended tools/sources.

Problem Statement, Target Users, and Use Cases

Problem: Users (researchers, analysts, students, support staff) face information overload in specialized domains. LLMs alone may hallucinate or lack up-to-date domain facts. A multi-agent RAG assistant can *automatically retrieve and synthesize* relevant knowledge from many sources, improving accuracy [3†L180-L188] [15†L118-L127] . It effectively acts like a smart “research assistant” or knowledge engine.

Target users: Academic researchers, R&D professionals, business analysts, legal/literature reviewers, and technical support teams. They need to query corporate knowledge bases, scientific literature, or mixed media (PDFs, docs) for precise answers.

Use cases: Summarizing academic papers; systematic literature reviews; domain-specific Q&A (e.g. “What does recent research say about CRISPR efficacy?”); customer support chatbots with product docs; enterprise search across manuals and logs; competitive intelligence and data extraction; medical/clinical insights; policy research. In each case, the assistant must retrieve authoritative sources (papers, databases, web APIs) and generate a concise answer with citations. [18†L72-L81] [61†L93-L102]

Market Need and Competitors

Modern enterprises and researchers demand faster, more reliable insights from their data. Existing AI tools often trade off depth for ease-of-use. Our solution addresses a clear need for scalable, accurate knowledge retrieval in specialized domains. Below is a comparison of five notable “AI research assistant” solutions:

Solution	Strengths	Weaknesses
Elicit	AI semantic search over 138M+ papers; generates structured research briefs with citations [18†L72-L81] [18†L106-L113] . Scalable (1000 papers) and validated accuracy.	Focuses on empirical/lit-review tasks; less conversational interactivity or external tool integration.
PaperGuide	All-in-one platform (AI search, lit review, writing, ref manager) over ~200M papers [20†L59-L68] . Answers with citations and deep analysis reports.	Newer tool; high complexity. Interface/algorithms still evolving; may be overkill for simple queries.
Iris.ai (Researcher Workspace)	Visual search and content analysis for R&D. Finds relevant documents, extracts data points, and summarizes thousands of papers [61†L93-L102] . Trainable on domain lexicons for R&D.	Geared towards heavily technical domains (chemistry, pharma, etc.). Enterprise-focused; may require paid licence.
SciSpace (Typeset)	AI academic writing assistant; autocomplete and citations from 280M+ papers [24†L29-L33] . Offers PDF Q&A ("Copilot"). Trusted by universities.	Primarily aids writing and comprehension. Not explicitly multi-agent; depends on a single LLM as copilot.
ChatPDF	"Chat with PDFs": upload any document and ask questions; built-in citations from the PDF [59†L148-L156] [59†L249-L258] . Easy to use (like ChatGPT for docs).	Limited to the uploaded file(s). No integration with external corpora or enterprise data. Not designed for multi-hop or multi-source queries.

Each competitor excels in a niche (scale, interface, citations) but lacks a modular multi-agent architecture. Our system combines the best of these: broad corpus access (like Elicit/PaperGuide), strong citation grounding (like Elicit/ChatPDF), and flexible agent-based orchestration for complex workflows.

System Architecture

The system consists of these core components:

- **Document Ingestion Module:** Collects and preprocesses data (PDFs, web pages, databases). Text is split into manageable chunks (e.g. ~1000 tokens) and embedded.
- **Vector Store (Weaviate):** Embeddings and associated metadata (source, title, etc.) are stored in Weaviate. Weaviate's hybrid search supports vector similarity and keyword filters [12†L61-L69] .
- **PostgreSQL DB:** Stores user data (accounts, sessions, chat history, feedback) and agent logs. It holds conversation transcripts and citation links.
- **Agent Orchestrator (LangChain/Agents SDK):** Manages "Agent" roles. When a query arrives, the orchestrator invokes a sequence of agents: e.g. a *Planner Agent* (optionally reformulates complex queries), one or more *Retrieval Agents* (query vector store, web APIs, etc.), and a *Generator Agent* (LLM that synthesizes the answer). Agents can collaborate iteratively (retrieve-evaluate-re-retrieve) [15†L118-L127] .

- **LLM (OpenAI API):** Used for embedding generation and answer synthesis (GPT-4/GPT-5 models). The LLM is provided with retrieved context to generate final answers with citations.
- **Memory Store:** Short-term: conversation history from PostgreSQL. Long-term: optional semantic memory (also stored in Weaviate or Redis) to remember user profile or prior findings [11†L197-L200] .
- **API Layer (Flask/FastAPI):** Exposes endpoints for clients (web/mobile). Manages authentication (API keys or JWT) and rate limiting.

【29†embed_image】 *Figure: Example multi-agent RAG architecture (from MA-RAG paper [9†L51-L59]).* In this design, specialized query-generation and retrieval agents feed a central “Query Execution” environment (vector DB, search) and a final generative agent synthesizes the answer [29†L639-L648] . We adopt a similar modular architecture where each agent has a defined role (e.g. database query agent, text retrieval agent). LangChain acts as the orchestration layer, allowing easy swapping of models (OpenAI, Cohere, etc.) and tools (Weaviate, web search) without rewriting logic [12†L61-L69] .

【58†embed_image】 *Figure: RAG pipeline (NVIDIA) showing document ingestion (left) and query (right) flows [58†] .* The ingestion path embeds documents into a vector DB; the query path embeds the user’s question, retrieves similar documents, and passes them (with the query) to the LLM to generate an answer [12†L61-L69] . The dotted lines indicate hybrid retrieval: Weaviate can filter by metadata and vector similarity simultaneously. Our data flow follows this paradigm: first preprocess and index, then at query time fetch and answer.

Database Schema & Folder Structure

Folder Structure: A typical code repository might look like:

```
/ (root)
├─ src/
│   ├── main.py           # Entry point, initializes services
│   ├── api/
│   │   ├── app.py        # FastAPI/Flask app
│   │   ├── endpoints.py  # Chat, session, user endpoints
│   ├── agents/
│   │   ├── base_agent.py # Base class for agents
│   │   ├── planner_agent.py
│   │   ├── retrieval_agent.py
│   │   └── answer_agent.py
│   ├── db/
│   │   ├── models.py     # SQLAlchemy models for Postgres
│   │   ├── schema.sql    # SQL DDL definitions
│   │   └── weaviate_store.py # Vector DB schema/config
│   ├── rag_pipeline.py   # Orchestration logic (ingestion and query)
│   ├── utils/
│   │   ├── embedding_utils.py
│   │   └── llm_utils.py
│   └── config.py         # Config (API keys, DB URLs)
```

```

├─ tests/                # Unit and integration tests
│   ├── test_agents.py
│   ├── test_api.py
│   └─ ...
├─ Dockerfile            # Docker build
├─ requirements.txt       # Python dependencies (langchain, weaviate-client,
openai, etc.)
└─ .github/workflows/    # CI/CD configs (e.g. GitHub Actions)

```

PostgreSQL Schema: We define tables for users, sessions, messages, and citations. For example (SQL definitions and sample records):

```

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name TEXT,
    email TEXT UNIQUE,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE sessions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    start_time TIMESTAMP,
    end_time TIMESTAMP
);

CREATE TABLE messages (
    id SERIAL PRIMARY KEY,
    session_id INTEGER REFERENCES sessions(id),
    role TEXT,          -- 'user' or 'assistant'
    content TEXT,
    timestamp TIMESTAMP DEFAULT NOW()
);

CREATE TABLE citations (
    id SERIAL PRIMARY KEY,
    message_id INTEGER REFERENCES messages(id),
    source TEXT,        -- e.g. paper title or URL
    snippet TEXT        -- quoted excerpt from source
);

```

Example records:

```

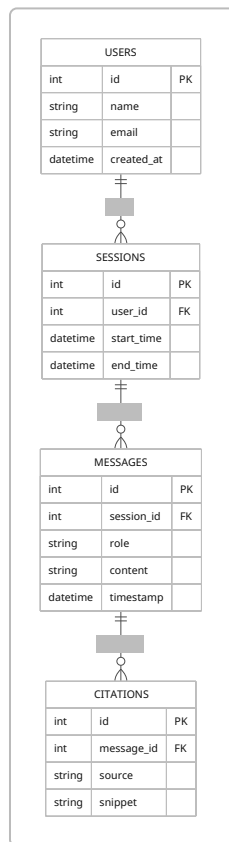
INSERT INTO users (id,name,email) VALUES (1,'Alice','alice@example.com');
INSERT INTO sessions (id,user_id,start_time) VALUES (101,1,'2026-06-09

```

```

10:00:00');
INSERT INTO messages (id,session_id,role,content) VALUES
  (1001,101,'user','What is agentic RAG?'),
  (1002,
101,'assistant','Agentic RAG integrates agents into RAG to iteratively retrieve
and validate context [15†L118-L127] .');
INSERT INTO citations (id,message_id,source,snippet) VALUES
  (501,1002,'IBM Think Article','"Agentic RAG turns retrieval into an
iterative process: the agent can retrieve, evaluate, re-retrieve, and validate
context..." [15†L123-L127] ');

```



This ER diagram shows our core entities and relations. The **USERS** table links to **SESSIONS**; each **SESSION** has multiple **MESSAGES**; each assistant message can have multiple **CITATIONS** pointing back to sources in the knowledge base. Weaviate's schema is separate (it stores indexed documents/chunks), so it's not shown in this ER diagram.

API Design

We expose a REST API (e.g. via FastAPI). Key endpoints (JSON I/O) include:

- **POST /api/session** – Create new chat session. Body: `{ "user_id": 1 }`. Returns: `{ "session_id": 101 }`.

- `POST /api/chat` – Send user message, get assistant response. Body: `{ "session_id": 101, "message": "Your question here" }`. Returns:

```
{
  "reply": "Generated answer with citations.",
  "sources": [
    { "source": "Document Title A", "snippet": "..." },
    { "source": "Document Title B", "snippet": "..." }
  ]
}
```

The server will log the user message and the assistant's reply (with citations) into the DB.

- `GET /api/history?session_id=101` – Retrieve full chat history. Returns list of `{role, content}` objects.
- `POST /api/feedback` – Collect user feedback (for evaluation). Body: `{ "session_id": 101, "message_id": 1002, "rating": 5, "notes": "Very helpful" }`.
- `POST /api/ingest` (optional) – Upload or register a new document/corpus for indexing. Body might be file content or URL.

Auth & Rate Limiting: Use API keys or JWT tokens. All endpoints require authentication. Rate limiting (e.g. 60 requests/min per user) is enforced to prevent abuse. For example, use FastAPI middleware or an API gateway (Nginx) to throttle.

Request/Response schemas are defined with clear JSON structures. For instance, `/api/chat` expects `{"session_id":INT,"message":STRING}` and returns `{"reply":STRING,"sources":[{"source:STRING,snippet:STRING},...]}`. Successful calls use HTTP 200; errors return 4xx.

Multi-Agent Design

The core innovation is a **multi-agent orchestration**. We define several agent types with distinct roles:

- **Planner Agent:** (Optional) Decomposes complex queries into sub-questions or identifies relevant topics. E.g., using few-shot prompts to clarify ambiguous terms.
- **Query Rewrite Agent:** Reformulates the user's question into an optimal search query (similar to Weaviate's Query Agent).
- **Retrieval Agents:** One or more agents, each querying a different source/tool:
 - *Vector DB Agent:* Uses Weaviate's similarity search (embedding retriever).
 - *Web Search Agent:* Calls a web search API (Google/Bing) if allowed.
- *Database Agent:* Runs SQL queries on a relational DB (if connected). These agents return relevant documents or data slices. Weaviate's hybrid search can simultaneously filter by metadata (e.g. date, author) and vector similarity [\[12†L61-L69\]](#).
- **Integrator/Reranker Agent:** (Optional) Given top-K results, this agent might use a small LLM or heuristic to rerank them by relevance or to filter out duplicates.

- **Answer Agent:** Takes the original user query and the retrieved contexts, and runs an LLM prompt to generate the final answer. It must cite sources from the retrieved set (similar to ChatPDF's citing [59†L148-L156] or Elicit's method [18†L200-L208]).

Orchestration: LangChain's Agent executor or OpenAI's Agents SDK can run these components in sequence. For example, upon a query:

1. (Optional) The Planner Agent checks if the question has multiple parts, splits it.
2. The system formulates a query for the Vector DB (using embedding) and separately possibly a web query.
3. Retrieval Agents fetch evidence (e.g. top-5 text chunks).
4. The Answer Agent uses a prompt like: "Given the question and the following excerpts (with citations), synthesize a concise answer with references."

Prompts & Templates: Each agent has a tailored prompt. For instance, the Answer Agent's system prompt stresses factuality and citation (e.g. "Use the sources explicitly; mark each fact with [source]"). Planner agents might use chain-of-thought style prompts (as in MA-RAG [9†L51-L59]) to reason step-by-step.

Failure Modes & Fallbacks: Agents must handle errors gracefully. For example, if the Retrieval Agent finds no relevant docs, the system can trigger a fallback: broaden the query (e.g. remove specific filters), or route to an alternative agent (web search). The IBM reference notes agentic systems are complex and can fail or hallucinate more often [11†L273-L281] , so we build in safeguards: timeouts for LLM calls, thresholding low-similarity results, and verification steps. If an agent's output is deemed low-confidence (e.g. LLM answer without supporting quotes), the system may respond: "I'm not sure; please provide more detail or check original sources."

By combining specialized agents, the system emulates a team of experts: one fetching data, one analyzing it, one writing answers. This modular approach (mirroring MA-RAG or orchestrated AI frameworks [9†L51-L59] [15†L181-L189]) makes the assistant flexible and extensible.

RAG Pipeline Details

Document Chunking & Embeddings: Incoming texts (docs, reports, transcripts) are split into overlapping chunks (LangChain's `CharacterTextSplitter` or `RecursiveTextSplitter`). This ensures each chunk fits the embedding model's context window. Various LangChain "CombineDocuments" chains (stuffing, map-reduce, refine, map-rerank) can be used for composing or summarizing multiple chunks [13†L94-L100] [13†L106-L113] . For example, large documents can be handled by *map-reduce*: first ask the LLM to summarize each chunk, then combine those summaries.

We use an embedding model (e.g. OpenAI's text-embedding-3-large or Cohere embeddings) to vectorize each chunk. Each vector is stored in Weaviate along with metadata (document ID, source, page).

Retrieval: When a query arrives, it's also embedded, and Weaviate is queried for nearest neighbors (semantic search). Weaviate's hybrid search can also use keyword filters (e.g. date range). We might retrieve top-K (e.g. 5–10) chunks. We could also use sparse retrievers (BM25) in parallel for speed. The retrieved chunks (with citations) form the context.

Reranking (optional): A lightweight LLM or a trainable reranker can score the top-N docs for relevance. For instance, run the query through a mini LLM with each candidate chunk to get a relevance score, then select the best few. This step improves precision on tricky queries.

Answer Generation: The final prompt to the LLM includes the user’s question and the retrieved contexts. The system message instructs the LLM to generate an answer that explicitly cites each fact (e.g. with footnote-style references). We follow Elicit’s approach: “support all claims with sentence-level citations from the sources” [18†L200-L208] . If multiple sources confirm a point, list both. If uncertain, ask the user or skip. The output JSON includes the answer text and a list of source-snippet pairs.

Citation Handling: Each source snippet is tagged with its origin. We ensure the LLM’s output embeds these citations in the response (for example, as [source] or numbered footnotes). This mirrors ChatPDF’s cited-sources feature [59†L148-L156] .

Chunking Strategy: Generally, moderate-sized chunks (~512–1024 tokens) strike a balance. Longer segments mean fewer retrievals but risk exceeding LLM context. We might use adaptive splitting: split by sentences near the max token size. We also index titles/headings as metadata to aid filtering.

Embedding Model: A high-quality embedding like OpenAI’s or Cohere’s large model works well. If cost is a concern, smaller models (DistilBERT, MiniLM) can be used for initial retrieval, with a higher-tier model for final reranking.

Retriever Types: Primarily, we use dense (vector) retrieval via Weaviate. For certain tasks, we could plug in domain-specific tools: e.g., a SQL agent for structured data, a calculator agent for numeric queries, or an image search agent for visual data. These become new *retrieval agents* in the multi-agent design.

Reranking & Validation: After the LLM generates an answer, we can optionally have a *validation agent* check factuality. For example, send the output back to a smaller model to spot-check that each cited snippet supports its claim. Any hallucinated fact (not found in sources) triggers a fallback (e.g. exclude it or re-query). This guards against LLM hallucinations.

Memory and Session Management

- **Short-Term Memory:** Each conversation’s messages (user and assistant) are stored in PostgreSQL with timestamps. We include recent history (e.g. last 10 turns) in the prompt context so the LLM can maintain dialogue coherence. LangChain’s conversation memory can also be used to automatically manage this history.
- **Long-Term Memory:** The system can optionally “remember” user-specific info or long-term research context. For instance, if the user frequently asks about “quantum computing,” we might store that as a topic vector in Weaviate or Redis. A memory module could be implemented using Weaviate as well, storing embeddings of salient Q&A pairs [11†L197-L200] . Agentic RAG advocates using a semantic cache for past queries and results [11†L197-L200] – effectively a history-based retriever.

- **Session Management:** Each session (chat) is given an ID. In the UI, users can see past sessions. A session ties to one user in the `users` table. We track session start/end times. Optionally, we may expire sessions after inactivity (e.g. 30 min idle).
- **Conversation State:** In-memory state (for the orchestrator) holds the current agent's intermediate outputs. If an agent fails, we can resume from the last safe checkpoint. All intermediate steps (e.g. Planner's breakdown, each agent's findings) are logged for auditing but typically not sent to the user.

Deployment Plan

- **Infrastructure:** Deploy as Docker containers. One container runs the API and agent orchestrator (Python). Another runs Weaviate (vector DB). Possibly a third for a message queue (e.g. Redis or RabbitMQ) if we decouple agents asynchronously. Kubernetes or AWS ECS can manage scaling. For 1,000 monthly users, a small cluster suffices (e.g. 2–4 vCPUs).
- **Cloud Services:** We can use Weaviate Cloud (SaaS) to avoid managing DB servers. For example, Weaviate's *Flex* plan starts at ~\$45/month (100k objects, pay-as-you-go) [41†L50-L58] . Alternatively, host Weaviate via its Docker image on AWS (t2.large or similar). The LLM is accessed via OpenAI's API, so no GPU infrastructure needed.
- **CI/CD:** Use GitHub Actions/GitLab CI. On push to main, run tests, build Docker images, and deploy to staging (e.g. AWS ECS or Heroku). On tagged release, deploy to production. Health checks (weekly synthetic queries) ensure uptime.
- **Monitoring & Logging:** Container logs (via Elasticsearch/CloudWatch) will record errors. Prometheus/Grafana can track usage metrics (requests per minute, latency).
- **Security:** Encrypt data at rest (Postgres, Weaviate volumes) and in transit (HTTPS/TLS on all endpoints). Use IAM roles for service accounts. Weaviate enterprise tiers support RBAC and VPC deployment [41†L207-L216] .
- **Cost Estimate (1,000 users/mo):**
 - *OpenAI API:* Assume 1,000 users × 10 queries each × ~500 tokens total per query = ~5M tokens. Using GPT-5.4 mini (\$0.75 per 1M input, \$4.50 per 1M output) [39†L73-L80] yields roughly: 5M input ⇒ \$3.75, 5M output ⇒ \$22.5. Even with overhead and caching, likely <\$50–100/mo. For larger models (e.g. GPT-4/5 flagship), costs could reach a few hundred dollars.
 - *Weaviate:* Flex plan ~\$45/mo for storage (10GB disk, 1GB RAM) [41†L50-L58] . Embedding (if using Weaviate Embeddings service) adds ~\$0.03–\$0.065 per million tokens [41†L164-L173] , but we can use OpenAI embeddings via API instead.
 - *Compute:* A modest server (2vCPU, 4GB RAM) ~ \$20–40/mo on AWS EC2. Container hosting and bandwidth ~ \$50/mo.
 - *Total:* On the order of \$200–300/month for 1000 users (mostly API calls and some DB hosting).

Evaluation Metrics and Testing Plan

We will rigorously evaluate both retrieval and generation quality, as well as system performance:

- **Retrieval Metrics:** Precision@K, Recall@K, and MRR on a held-out set of queries with known relevant docs. This measures how often the agent fetches correct context. For example, if we have test questions with ground-truth answers in our corpus, we check if the relevant chunks were retrieved.
- **Generation Metrics:** For QA tasks, use exact match (EM) or F1 against reference answers. For summarization, use ROUGE/BLEU scores against gold summaries. Crucially, we measure **factual accuracy/faithfulness**: what fraction of answer statements are supported by the sources [49†L49-L57] . Tools like FactCC or manual spot-checking can estimate hallucination rate. We also ensure citations match the content.
- **Latency:** Measure average response time from API call to answer (LLM + retrieval). Goal might be <3 seconds. Use Prometheus to track p95 latency.
- **Robustness:** Test failure cases: ambiguous queries, no relevant docs, contradictory sources. See if the system gracefully declines or reformulates.
- **Load Testing:** Simulate 1000 concurrent users using tools like Locust. Ensure the service handles QPS and respects rate limits.
- **User Testing:** Conduct user studies or surveys. E.g. have domain experts rate answer helpfulness and accuracy. Check UX: clarity of citations, interface usability.
- **Hallucination Rate:** Specifically track hallucinations by verifying a sample of answers against sources. A low hallucination rate is a key requirement.

We will iterate based on these metrics. For example, if accuracy is low, we may augment the RAG pipeline (more vectors, better prompts). If latency is high, we may cache frequent queries or optimize prompts. Tracking both quantitative scores and qualitative feedback is essential [49†L49-L57] .

Security, Privacy, and Compliance

- **Data Security:** All data in transit is encrypted (HTTPS). Databases encrypt at rest. Use strong IAM (no hardcoded keys). Weaviate and Postgres are inside a VPC/secured network.
- **Access Control:** Users authenticate (JWT/API keys). Weaviate supports RBAC; we set permissions so only the application can query the index. Logs record admin actions.
- **Privacy & Compliance:** If handling personal or sensitive data, we must comply with GDPR/HIPAA as needed. On Weaviate's Premium plan, HIPAA and encrypted customer volumes are supported [41†L208-L216] . For ultimate privacy, one could deploy everything on-premises or use on-device LLMs. For example, LM-Kit (a .NET local AI library) enables running agents and models entirely on a customer's own servers (with encrypted models) [53†L143-L152] . This avoids sending any data to third-party APIs. In our hybrid scenario, we rely on OpenAI for LLM inference: OpenAI's enterprise policies state they do not use data for training, and allow deletion on request. Still, we should avoid sending highly sensitive user data to the API.
- **Content Filtering:** Implement filters to block illegal or disallowed content in queries or outputs. Use OpenAI's moderation endpoint or keyword checks.

- **Auditability:** We log all agent queries and LLM outputs (for a limited time) to allow audits. Citation chains provide traceability of each answer's provenance.

By design, our stack uses established compliant services (AWS, Weaviate Enterprise, OpenAI Enterprise) and supports on-prem/self-hosted deployment for maximum control.

60-Day Implementation Roadmap (by Day)

1. **Day 1-5 – Kickoff & Environment Setup:** Project scaffolding, repo and CI pipeline. Define requirements, choose models. Set up Postgres and Weaviate instances (locally or cloud).
2. **Day 6-10 – Basic Ingestion Pipeline:** Implement text loaders (PDF, plain text). Develop document splitter and test embeddings + storage in Weaviate. Index a small corpus for prototype.
3. **Day 11-15 – Retrieval & QA Chain:** Build a simple LangChain (or custom) RAG chain: query→retrieve top-K→LLM answer. Hardcode prompts to ensure citations. Test on example queries.
4. **Day 16-20 – API and Chat Interface:** Create minimal API endpoints (/session , /chat). Develop a basic CLI or web UI. Validate end-to-end: client sends question, gets answer with sources.
5. **Day 21-25 – Multi-Agent Orchestration:** Refactor the single chain into multiple agents. Add a Planner (LLM chain-of-thought) and a separate Retrieval Agent. Use LangChain's Agents or flow control. Ensure agents can pass context.
6. **Day 26-30 – Advanced Retrieval:** Integrate hybrid search or an external search API. Implement reranking: use LLM to score retrieved docs. Add caching of embeddings and query results.
7. **Day 31-35 – Memory & Context:** Add conversation memory (store and recall chat history from Postgres). Implement long-term memory prototype (e.g. tag user topics in Weaviate).
8. **Day 36-40 – Error Handling & Refinement:** Develop fallback logic: handle no-results or LLM failure. Improve prompts for few-shot chain-of-thought (MA-RAG style [9†L51-L59]). Test failure scenarios.
9. **Day 41-45 – Database Schema & Management:** Finalize PostgreSQL models and migrations. Write unit tests for DB interactions (sessions, messages, citations). Add sample seed data.
10. **Day 46-50 – Performance Tuning:** Benchmark latency and throughput. Optimize vector DB parameters (HNSW dimensions, ef). Enable request batching or caching. Add pagination if needed.
11. **Day 51-55 – Security & Deployment Prep:** Containerize services (Dockerfile). Configure HTTPS and auth middleware. Prepare deployment scripts (Docker Compose, Kubernetes manifests). Write infrastructure docs.
12. **Day 56-58 – Testing & QA:** Run integration tests: sample Q&A benchmarks, rate-limit tests. User acceptance testing with beta users. Fix bugs.
13. **Day 59-60 – Finalize & Launch:** Deploy to production. Monitor initial load, ensure metrics are collected. Hand over documentation and developer guide.

Prioritized MVP Feature List:

1. Core RAG Chat (single agent, vector DB retrieval, answer with citations).
2. Basic multi-step queries (allow simple follow-up handling).
3. Session management & history (store chats).
4. Admin UI or logs viewer.
5. Basic user feedback/rating system.

Features like robust multi-agent workflows, web tool integration, advanced memory, UI polish, and enterprise security (SSO, audit logs) can follow in later iterations.

Recommended Tools and Sources

- **LangChain (and OpenAI Agents SDK):** Orchestration of LLM chains and agents [12†L61-L69] [13†L94-L103] .
- **Weaviate:** Enterprise-grade vector DB with hybrid search and built-in services [12†L61-L69] [41†L164-L173] . (Alternatively Pinecone or Milvus).
- **OpenAI API:** GPT-4/GPT-5 models for embeddings and generation (documented pricing [39†L73-L80]).
- **LM-Kit:** If self-hosted LLMs needed for privacy (supports encrypted models) [53†L143-L152] .
- **Hugging Face Transformers:** For any open-source models (e.g. Llama or Mistral) if offline inference is desired.
- **Docker/Kubernetes & CI/CD:** GitHub Actions, Docker Compose for deployment.
- **Key Papers/Blogs:** The IBM Think piece on Agentic RAG [3†L180-L188] [11†L273-L281] , MA-RAG arXiv [9†L51-L59] , Weaviate Agentic RAG blog [15†L118-L127] , NVIDIA's RAG tutorial [57†L131-L140] , and the RAG evaluation survey [49†L49-L57] . These offer guidance on architecture and evaluation.

By leveraging these tools and a phased roadmap, we can build a production-grade multi-agent RAG research assistant that is accurate, extensible, and ready for real-world demands. Each component and decision is guided by current best practices and primary sources.
